CO1 - Assessment Tool 1

Name : GUNA P
Reg NO: 192424277
Course code: DSA0503
Course Name: Query Processing for Data Science

JSON Nested Objects:

AIM:

To extract the employee name, city, and state from a nested JSON object and convert the extracted data into a Pandas DataFrame.

Explanation:

JSON (JavaScript Object Notation) is used to store and exchange data. In this example, the employee details contain a nested address object with city and state information. The program reads the JSON data, extracts the required fields, stores them in a list of dictionaries, and converts the list into a Pandas DataFrame for easy display.

Algorithm:

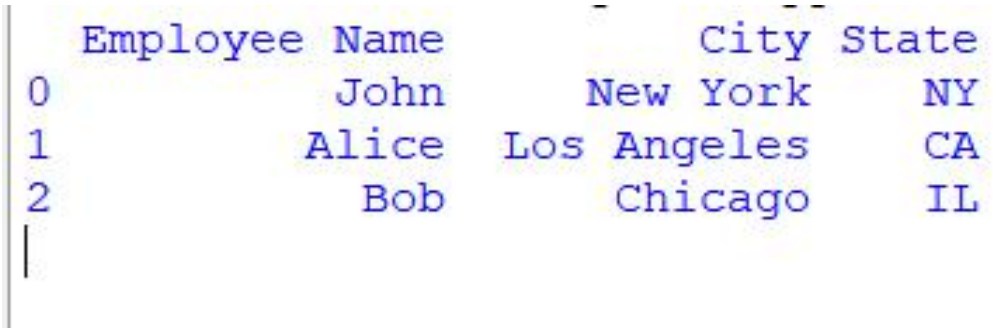
- ✧ Start the program.
- ✧ Import the required libraries: json and pandas.
- ✧ Store the employee details in a JSON string.
- ✧ Convert the JSON string into a Python dictionary using json.loads().
- ✧ Create an empty list to store the extracted employee details.
- ✧ Traverse through each employee in the employees list.
- ✧ Extract the following fields:
- ✧ Employee Name
- ✧ City (from the nested address object)
- ✧ State (from the nested address object)
- ✧ Store the extracted values as a dictionary in the list.
- ✧ Convert the list of dictionaries into a Pandas DataFrame.
- ✧ Display the DataFrame.
- ✧ Stop the program.

CODE:

```
import json
import pandas as pd

json_data = '''
{
```

```
    "employees": [
        {
            "name": "John",
            "address": {
                "city": "New York",
                "state": "NY"
            }
        },
        {
            "name": "Alice",
            "address": {
                "city": "Los Angeles",
                "state": "CA"
            }
        },
        {
            "name": "Bob",
            "address": {
                "city": "Chicago",
                "state": "IL"
            }
        }
    ]
}
'''

data = json.loads(json_data)

employee_details = []

for employee in data["employees"]:
    name = employee["name"]
    city = employee["address"]["city"]
    state = employee["address"]["state"]

    employee_details.append({
        "Employee Name": name,
        "City": city,
        "State": state
    })

df = pd.DataFrame(employee_details)

print(df)
```

OUTPUT:



	Employee Name	City	State
0	John	New York	NY
1	Alice	Los Angeles	CA
2	Bob	Chicago	IL

Conclusion:

The nested JSON data was successfully processed by extracting the employee name, city, and state from the nested address object. The extracted information was then converted into a Pandas DataFrame, making the data organized, structured, and easy to view and analyze.